

<https://github.com/Misbah2526>

DATA STRUCTURES (LAB)

LAB ASSIGNMENT 01



By

MISBAH KHALID

FA25-BCS-005

Submitted to,

Mr. BILAL BUKHARI

OCTOBER 4TH, 2026

**Department of Computer Sciences
COMSATS University Islamabad,
Attock Campus**

SINGLY LINK LIST

TASK 01

CODE:

```
#include <iostream> // input output library
using namespace std; // standard name space

// Node structure
struct NodeType {
    int data; // store the value of node
    NodeType *next; // store the address of the next node
};

// create 2 pointers for linklist
NodeType *first = NULL, *last = NULL; // first point to first node last points to last node

// Insert at end
void insert_end(int value) {
    NodeType *p = new NodeType; // create new node dynamically
    p->data = value; // store given value to new node
    p->next = NULL; // new node will be the last node so it is null
    // check if the link list is empty
    if (first == NULL) {
        first = last = p; // both pointers point to new node
    } else {
        last->next = p; // connect current last node to the new node
        last = p;
    }
}

// Recursive display in reverse
void displayReverseRecursive(NodeType *p) {
    if (p == NULL) return;
    displayReverseRecursive(p->next); // go to end first
    cout << p->data << " "; // print on way back
}

// Iterative reverse display using recursion internally
void displayReverse() {
    cout << "List in reverse: ";
    displayReverseRecursive(first);
    cout << endl;
}

int main() {
    insert_end(100); // oldest
    insert_end(200);
    insert_end(300);
}
```

```

insert_end(400); // newest

displayReverse(); // 400 300 200 100

return 0;
}

```

OUTPUT:

```

C:\Users\Discount Laptop\De: × + ▾
List in reverse: 400 300 200 100
-----
Process exited after 0.2907 seconds with return value 0
Press any key to continue . . . |

```

TASK 02

CODE

```

#include <iostream>
using namespace std;

struct NodeType {
    int data;
    NodeType *next;
};

NodeType *first = NULL, *last = NULL; // first last pointers empty
// function to insert at end
void insert_end(int value) {
    NodeType *p = new NodeType;
    p->data = value;
    p->next = NULL;
    if (first == NULL) first = last = p;
    else { last->next = p; last = p; }
}

// Reverse list by changing links only (addresses remain same)
void reverseList() {
    NodeType *prev = NULL;
    NodeType *current = first;
    NodeType *nextNode = NULL;

```

```

last = first; // old first becomes new last

while (current != NULL) { //continue until p reaches null
    nextNode = current->next;
    current->next = prev;
    prev = current;
    current = nextNode;
}
first = prev;
}

void display() {
    NodeType *p = first;
    while (p != NULL) {
        cout << p->data << " -> ";
        p = p->next;
    }
    cout << "NULL" << endl;
}

int main() {
    insert_end(1); insert_end(2); insert_end(3);
    insert_end(4); insert_end(5);

    cout << "Original: "; display();
    reverseList();
    cout << "Reversed: "; display();

    return 0;
}

```

OUTPUT:

```

29
30
31
32 Original: 1 -> 2 -> 3 -> 4 -> 5 -> NULL
33 Reversed: 5 -> 4 -> 3 -> 2 -> 1 -> NULL
34
35 -----
36
37 Process exited after 0.3135 seconds with return value 0
38 Press any key to continue . . .
39
40
41
42
43
44
45

```

TASK 03

CODE

```
#include <iostream>
using namespace std;

// Structure for creating a node
struct NodeType {
    int data;          // Stores the value of the node
    NodeType *next;    // Stores the address of the next node
};

// Pointers to the first and last node
NodeType *first = NULL, *last = NULL;

// Function to insert a new node at the end
void insert_end(int value) {

    // Create a new node dynamically
    NodeType *p = new NodeType;

    // Store the given value in the new node
    p->data = value;

    // New node will be the last node, so next is NULL
    p->next = NULL;

    // If the list is empty
    if (first == NULL)
        first = last = p; // Both first and last point to new node

    // If the list is not empty
    else {
        last->next = p; // Connect old last node to new node
        last = p;      // Make new node the last node
    }
}

// Function to remove duplicate values
// No extra data structure is used
void removeDuplicates() {

    // Start from the first node
    NodeType *current = first;

    // Continue while current and its next node exist
```

```

while (current != NULL && current->next != NULL) {

    // Runner starts from the current node
    NodeType *runner = current;

    // Check all nodes after current
    while (runner->next != NULL) {

        // If the next node has the same value as current
        if (runner->next->data == current->data) {

            // Store the duplicate node
            NodeType *dup = runner->next;

            // Skip the duplicate node
            runner->next = runner->next->next;

            // If duplicate was the last node
            if (dup == last)
                last = runner; // Update last pointer

            // Delete duplicate node from memory
            delete dup;
        }

        // If it is not a duplicate
        else {
            // Move runner to the next node
            runner = runner->next;
        }
    }

    // Move current to the next node
    current = current->next;
}
}

```

```

// Function to display the linked list
void display() {

    // Start from the first node
    NodeType *p = first;

    // Traverse until the end of the list
    while (p != NULL) {

        // Print the data of the current node
        cout << p->data << " -> ";
    }
}

```

```

        // Move to the next node
        p = p->next;
    }

    // Show NULL at the end of the list
    cout << "NULL" << endl;
}

int main() {

    // Insert values into the linked list
    insert_end(10);
    insert_end(20);
    insert_end(10);
    insert_end(30);
    insert_end(20);
    insert_end(40);
    insert_end(30);

    // Display the original list
    cout << "Original: ";
    display();

    // Remove duplicate values
    removeDuplicates();

    // Display the list after removing duplicates
    cout << "After removing duplicates: ";
    display();

    // End the program
    return 0;
}

```

OUTPUT:

```
C:\Users\Discount Laptop\De: × + v
Original: 10 -> 20 -> 10 -> 30 -> 20 -> 40 -> 30 -> NULL
After removing duplicates: 10 -> 20 -> 30 -> 40 -> NULL

-----
Process exited after 0.318 seconds with return value 0
Press any key to continue . . . |
```

TASK 04

CODE

```
#include <iostream>
using namespace std;

// Structure for a linked list node
struct NodeType {
    int data;          // Stores the value
    NodeType *next;    // Stores address of the next node
};

// Pointers to the first and last node
NodeType *first = NULL, *last = NULL;

// Function to insert a node at the end
void insert_end(int value) {

    // Create a new node
    NodeType *p = new NodeType;

    // Store the value in the node
    p->data = value;

    // New node is currently the last node
    p->next = NULL;

    // If the list is empty
    if (first == NULL)
        first = last = p; // First and last point to new node

    // If the list is not empty
    else {
```

```

        last->next = p;    // Connect last node to new node
        last = p;         // Make new node the last node
    }
}

```

```

// Function to create a loop in the linked list
void createLoop(int pos) {

```

```

    // Start from the first node
    NodeType *loopNode = first;

```

```

    // Move to the node at the given position
    for (int i = 0; i < pos && loopNode != NULL; i++)
        loopNode = loopNode->next;

```

```

    // If the required node exists
    if (loopNode != NULL)

```

```

        // Connect last node back to that node
        last->next = loopNode;
    }
}

```

```

// Floyd's Cycle Detection Algorithm
// Also called Tortoise and Hare algorithm
bool detectLoop() {

```

```

    // Slow pointer moves one step at a time
    NodeType *slow = first;

```

```

    // Fast pointer moves two steps at a time
    NodeType *fast = first;

```

```

    // Continue while pointers can move
    while (slow != NULL && fast != NULL && fast->next != NULL) {

```

```

        // Move slow pointer one step
        slow = slow->next;

```

```

        // Move fast pointer two steps
        fast = fast->next->next;

```

```

        // If both pointers meet, a loop exists
        if (slow == fast)
            return true;
    }
}

```

```

// If pointers never meet, there is no loop
return false;

```

```

}

int main() {

    // Insert values into the linked list
    insert_end(1);
    insert_end(2);
    insert_end(3);
    insert_end(4);
    insert_end(5);

    // Create a loop from the last node
    // back to node at index 2 (value 3)
    createLoop(2);

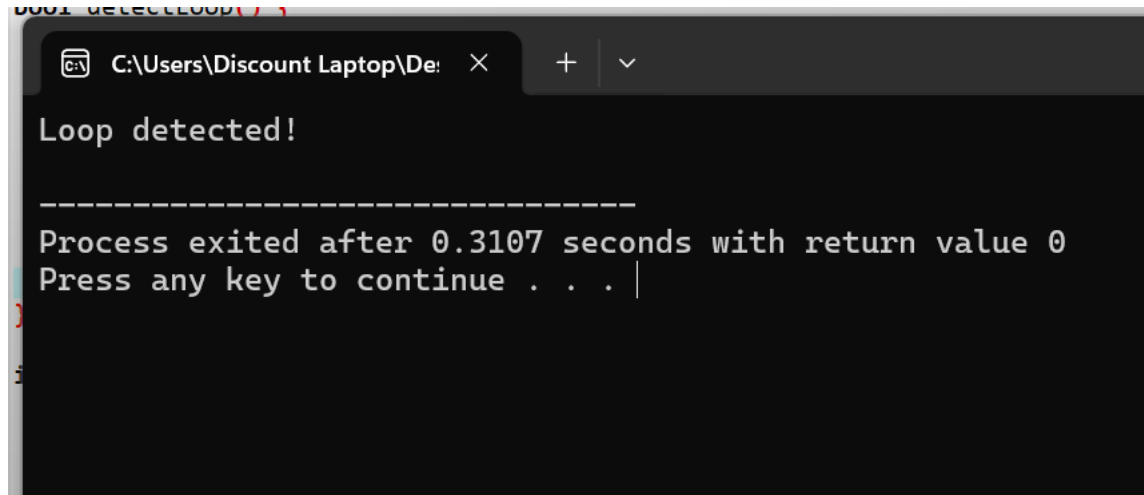
    // Check whether the linked list contains a loop
    if (detectLoop())
        cout << "Loop detected!" << endl;

    // If no loop is found
    else
        cout << "No loop found." << endl;

    return 0; // End the program
}

```

OUTPUT:



```

C:\Users\Discount Laptop\De:
Loop detected!
-----
Process exited after 0.3107 seconds with return value 0
Press any key to continue . . . |

```

TASK 05

CODE

```

#include <iostream>
using namespace std;

// Structure for a node
struct Node {

```

```

    int data;    // Stores the value of the node
    Node* next;  // Stores the address of the next node
};

// Function to find the middle node
Node* findMiddle(Node* head) {

    // If the list is empty, return NULL
    if (head == NULL)
        return NULL;

    // slow moves one node at a time
    Node* slow = head;

    // fast moves two nodes at a time
    Node* fast = head;

    // Move slow by 1 step and fast by 2 steps
    // This gives the left middle for an even-sized list
    while (fast->next != NULL && fast->next->next != NULL) {

        slow = slow->next;    // Move slow one step

        fast = fast->next->next; // Move fast two steps
    }

    // Return the middle node
    return slow;
}

// Function to insert a new node at the end
void insertEnd(Node*& head, int value) {

    // Create a new node dynamically
    Node* newNode = new Node;

    // Store the given value in the new node
    newNode->data = value;

    // New node will be the last node
    newNode->next = NULL;

    // If the list is empty, new node becomes the head
    if (head == NULL) {
        head = newNode;
        return;
    }

    // Temporary pointer starts from the first node
    Node* temp = head;

```

```

// Move temp until it reaches the last node
while (temp->next != NULL) {
    temp = temp->next;
}

// Connect the last node to the new node
temp->next = newNode;
}

int main() {

    // Initially the linked list is empty
    Node* head = NULL;

    // Insert values into the linked list
    insertEnd(head, 1);
    insertEnd(head, 2);
    insertEnd(head, 3);
    insertEnd(head, 4);
    insertEnd(head, 5);

    // Find the middle node
    Node* middle = findMiddle(head);

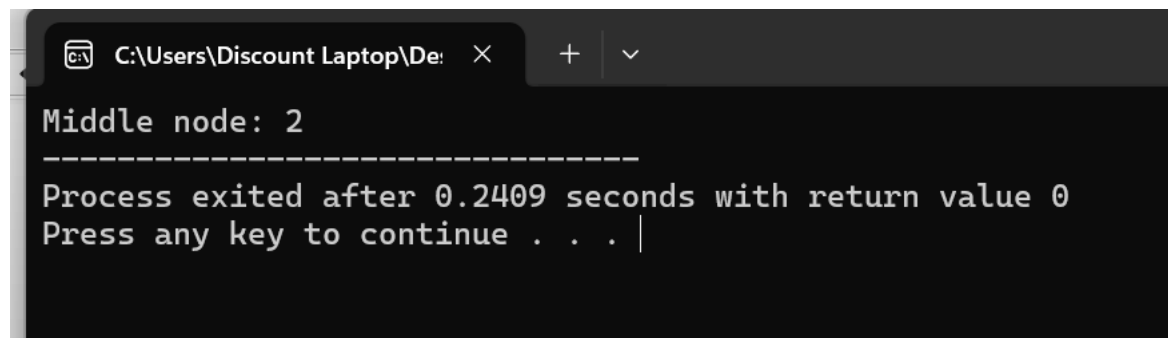
    // Check if a middle node exists
    if (middle != NULL)

        // Display the value of the middle node
        cout << "Middle node: " << middle->data;

    return 0; // End the program
}

```

OUTPUT



```

C:\Users\Discount Laptop\De:
Middle node: 2
-----
Process exited after 0.2409 seconds with return value 0
Press any key to continue . . . |

```

TASK 06

CODE

```

#include <iostream>
using namespace std;

```

```

// Structure for a node
struct NodeType {
    int data;          // Stores the price/value
    NodeType *next;    // Stores address of the next node
};

// Pointers for the first and last node
NodeType *first = NULL, *last = NULL;

// Function to insert a node at the end
void insert_end(int value) {

    // Create a new node
    NodeType *p = new NodeType;

    // Store the value in the new node
    p->data = value;

    // New node is currently the last node
    p->next = NULL;

    // If the list is empty
    if (first == NULL)
        first = last = p;    // New node becomes first and last

    else {
        last->next = p;      // Connect old last node to new node
        last = p;           // Make new node the last node
    }
}

// Function to separate even and odd values
void separateEvenOdd(NodeType *&evenHead, NodeType *&oddHead) {

    // Initially both lists are empty
    evenHead = oddHead = NULL;

    // Pointers to keep track of the last node
    // in the even and odd lists
    NodeType *evenTail = NULL;
    NodeType *oddTail = NULL;

    // Start from the first node of the original list
    NodeType *p = first;

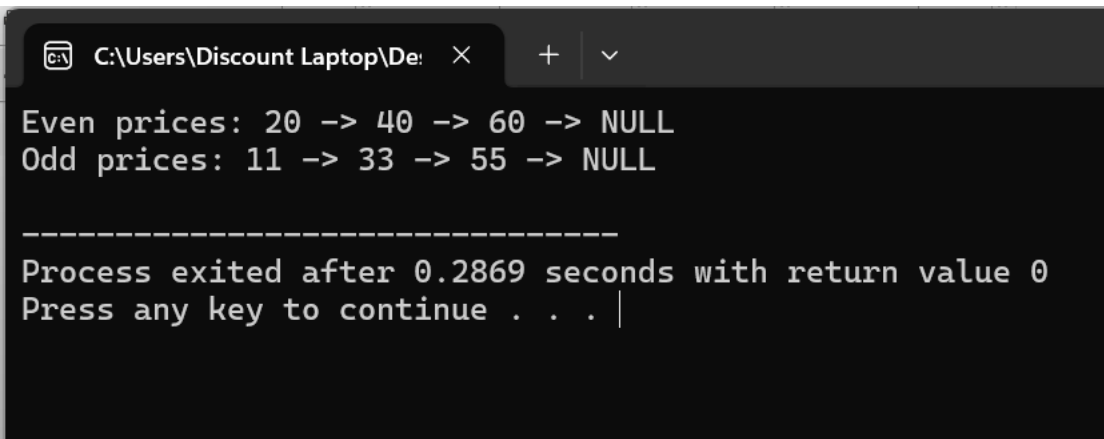
    // Continue until the end of the list
    while (p != NULL) {

```

```
// Save the next node before changing the link
NodeType *nextNode = p->next;

// Detach the current node from the original list
p->next =
```

OUTPUT:



```
C:\Users\Discount Laptop\De: X + v

Even prices: 20 -> 40 -> 60 -> NULL
Odd prices: 11 -> 33 -> 55 -> NULL

-----

Process exited after 0.2869 seconds with return value 0
Press any key to continue . . . |
```

TASK 07

CODE

```
#include <iostream>
using namespace std;

struct NodeType {
    int data;
    NodeType *next;
};

NodeType *first = NULL, *last = NULL;

void insert_end(int value) {
    NodeType *p = new NodeType;
    p->data = value;
    p->next = NULL;
    if (first == NULL) first = last = p;
    else { last->next = p; last = p; }
}
```

```

// Reverse a segment from start to end
NodeType* reverseSegment(NodeType *start, NodeType *end) {
    NodeType *prev = NULL;
    NodeType *curr = start;
    NodeType *stop = end->next;

    while (curr != stop) {
        NodeType *nextNode = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nextNode;
    }
    return prev;
}

// Reverse halves separately
void reverseHalves() {
    if (first == NULL || first->next == NULL) return;

    // Find middle
    NodeType *slow = first, *fast = first;
    NodeType *prevSlow = NULL;

    while (fast != NULL && fast->next != NULL) {
        prevSlow = slow;
        slow = slow->next;
        fast = fast->next->next;
    }

    NodeType *firstHalfStart = first;
    NodeType *firstHalfEnd = prevSlow;
    NodeType *secondHalfStart = slow;

    NodeType *secondHalfEnd = secondHalfStart;
    while (secondHalfEnd->next != NULL)
        secondHalfEnd = secondHalfEnd->next;

    // Reverse each half
    NodeType *newFirstHead = reverseSegment(firstHalfStart, firstHalfEnd);
    NodeType *newSecondHead = reverseSegment(secondHalfStart, secondHalfEnd);

    // Connect
    firstHalfStart->next = newSecondHead;
    first = newFirstHead;
}

void display() {
    NodeType *p = first;
    while (p != NULL) {
        cout << p->data;
    }
}

```

```

        if (p->next != NULL) cout << " -> ";
        p = p->next;
    }
    cout << endl;
}

int main() {
    for (int i = 1; i <= 8; i++) insert_end(i);

    cout << "Original: "; display();
    reverseHalves();
    cout << "After: "; display();

    return 0;
}

```

OUTPUT;

```

C:\Users\Discount Laptop\De:
Original: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8
After: 4 -> 3 -> 2 -> 1 -> 8 -> 7 -> 6 -> 5

-----
Process exited after 0.2861 seconds with return value 0
Press any key to continue . . .

```

TASK 08

CODE

```

#include <iostream>
using namespace std;

// Structure for a node
struct NodeType {
    int data;          // Stores the stamp/design number
    NodeType *next;    // Points to the next node
};

// Pointers to the first and last node
NodeType *first = NULL, *last = NULL;

```

```

// Function to insert a node at the end
void insert_end(int value) {

    // Create a new node
    NodeType *p = new NodeType;

    // Store the value
    p->data = value;

    // New node will be the last node
    p->next = NULL;

    // If list is empty
    if (first == NULL)
        first = last = p;    // New node becomes first and last

    else {
        last->next = p;    // Connect last node to new node
        last = p;        // Update last pointer
    }
}

```

```

// Function to remove duplicate values
void removeDuplicates() {

    // Start from the first node
    NodeType *current = first;

    // Continue while current and next node exist
    while (current != NULL && current->next != NULL) {

        // Runner starts from the current node
        NodeType *runner = current;

        // Check all nodes after current
        while (runner->next != NULL) {

            // Check if the next node is a duplicate
            if (runner->next->data == current->data) {

                // Store the duplicate node
                NodeType *dup = runner->next;

                // Skip the duplicate node
                runner->next = runner->next->next;

                // If duplicate was the last node
                if (dup == last)
                    last = runner;    // Update last pointer
            }
        }
    }
}

```

```

        // Delete duplicate node from memory
        delete dup;
    }

    else {
        // Move runner to the next node
        runner = runner->next;
    }
}

// Move current to the next node
current = current->next;
}
}

```

```

// Function to display the linked list
void display() {

```

```

    // Start from the first node
    NodeType *p = first;

    // Traverse the list
    while (p != NULL) {

        // Display current node
        cout << p->data << " -> ";

        // Move to the next node
        p = p->next;
    }

    // Show the end of the list
    cout << "NULL" << endl;
}

```

```

int main() {

    // Insert values into the linked list
    insert_end(101);
    insert_end(202);
    insert_end(101);
    insert_end(303);
    insert_end(202);
    insert_end(404);
    insert_end(303);

    // Display original list

```

```

    cout << "Original: ";
    display();

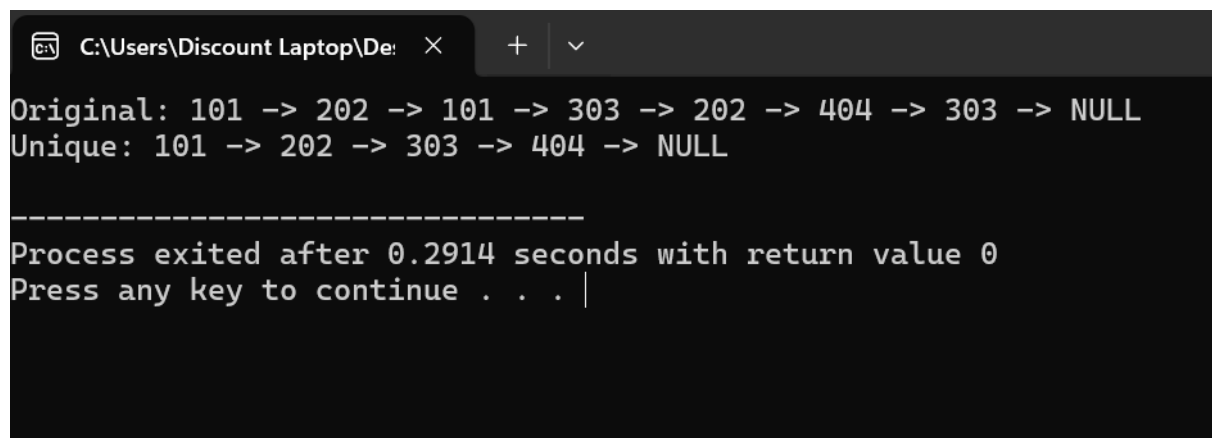
    // Remove all duplicate nodes
    removeDuplicates();

    // Display list after removing duplicates
    cout << "Unique: ";
    display();

    return 0;    // End the program
}

```

OUTPUT:



```

Original: 101 -> 202 -> 101 -> 303 -> 202 -> 404 -> 303 -> NULL
Unique: 101 -> 202 -> 303 -> 404 -> NULL

-----
Process exited after 0.2914 seconds with return value 0
Press any key to continue . . . |

```

TASK 09

CODE

```

#include <iostream>
using namespace std;

// Structure for a node
struct NodeType {
    int data;        // Stores the book ID
    NodeType *next;  // Points to the next node
};

// Pointers to the first and last node
NodeType *first = NULL, *last = NULL;

// Function to insert a node at the end
void insert_end(int value) {

    // Create a new node
    NodeType *p = new NodeType;

```

```

// Store the value in the node
p->data = value;

// New node is the last node
p->next = NULL;

// If the list is empty
if (first == NULL)
    first = last = p;    // New node becomes first and last

else {
    last->next = p;    // Connect old last node to new node
    last = p;        // Update last pointer
}
}

```

```

// Function to delete all occurrences of a given book ID
void deleteAllInstances(int bookID) {

```

```

    // Delete matching nodes from the beginning
    while (first != NULL && first->data == bookID) {

```

```

        // Store the first node temporarily
        NodeType *temp = first;

```

```

        // Move first to the next node
        first = first->next;

```

```

        // Delete the old first node
        delete temp;
    }

```

```

    // If the list becomes empty
    if (first == NULL) {
        last = NULL;    // Reset last pointer
        return;        // Stop the function
    }

```

```

    // Start checking from the first node
    NodeType *current = first;

```

```

    // Continue while there is a next node
    while (current != NULL && current->next != NULL) {

```

```

        // Check if the next node contains the given ID
        if (current->next->data == bookID) {

```

```

            // Store the node to be deleted

```

```

    NodeType *temp = current->next;

    // Skip the node that we want to delete
    current->next = current->next->next;

    // If deleted node was the last node
    if (temp == last)
        last = current;    // Update last pointer

    // Delete the node from memory
    delete temp;
}

else {
    // Move to the next node
    current = current->next;
}
}
}

```

```

// Function to display the linked list
void display() {

```

```

    // Start from the first node
    NodeType *p = first;

    // Traverse the list
    while (p != NULL) {

        // Display the book ID
        cout << p->data << " -> ";

        // Move to the next node
        p = p->next;
    }

    // Show the end of the list
    cout << "NULL" << endl;
}

```

```

int main() {

    // Insert book IDs into the linked list
    insert_end(5);
    insert_end(10);
    insert_end(5);
    insert_end(20);
    insert_end(5);
}

```

```

insert_end(30);
insert_end(5);

// Display the original list
cout << "Original: ";
display();

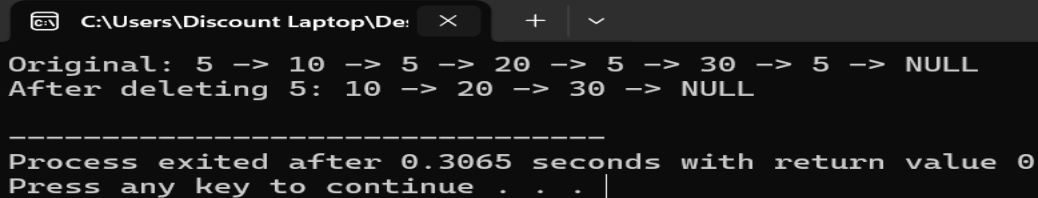
// Delete all occurrences of book ID 5
deleteAllInstances(5);

// Display the list after deletion
cout << "After deleting 5: ";
display();

return 0;    //

```

OUTPUT:



```

Original: 5 -> 10 -> 5 -> 20 -> 5 -> 30 -> 5 -> NULL
After deleting 5: 10 -> 20 -> 30 -> NULL

-----
Process exited after 0.3065 seconds with return value 0
Press any key to continue . . . |

```

TASK 10

CODE

```

#include <iostream>
using namespace std;

// Structure for a node
struct NodeType {
    int data;        // Stores the value
    NodeType *next;  // Points to the next node
};

// Pointers to the first and last node
NodeType *first = NULL, *last = NULL;

// Function to insert a node at the end
void insert_end(int value) {

    // Create a new node
    NodeType *p = new NodeType;

    // Store the value
    p->data = value;

```

```

// New node will be the last node
p->next = NULL;

// If list is empty
if (first == NULL)
    first = last = p;    // New node becomes first and last

else {
    last->next = p;      // Connect last node to new node
    last = p;           // Update last pointer
}
}

// Function to swap every two consecutive nodes
void swapPairs() {

    // If list has 0 or 1 node, there is nothing to swap
    if (first == NULL || first->next == NULL)
        return;

    // Create a dummy node
    // It makes swapping the first pair easier
    NodeType *dummy = new NodeType;

    // Dummy points to the first node
    dummy->next = first;

    // prev points to the node before the pair
    NodeType *prev = dummy;

    // Continue while two nodes are available
    while (prev->next != NULL && prev->next->next != NULL) {

        // Store the first node of the pair
        NodeType *firstNode = prev->next;

        // Store the second node of the pair
        NodeType *secondNode = firstNode->next;

        // ----- Swap the two nodes -----

        // First node points to the node after second
        firstNode->next = secondNode->next;

        // Second node points to the first node
        secondNode->next = firstNode;
    }
}

```

```

// Previous node points to the second node
prev->next = secondNode;

// Move prev to the first node
// because it is now after the second node
prev = firstNode;
}

// Update first pointer
first = dummy->next;

// Delete the dummy node
delete dummy;

// Update the last pointer
last = first;

// Move last to the final node
while (last != NULL && last->next != NULL)
    last = last->next;
}

// Function to display the linked list
void display() {

    // Start from the first node
    NodeType *p = first;

    // Traverse the list
    while (p != NULL) {

        // Display the data
        cout << p->data;

        // Print arrow if another node exists
        if (p->next != NULL)
            cout << " -> ";

        // Move to the next node
        p = p->next;
    }

    // Show the end of the list
    cout << " -> NULL" << endl;
}

```

```

int main() {

    // Insert values into the linked list
    insert_end(1);
    insert_end(2);
    insert_end(3);
    insert_end(4);
    insert_end(5);
    insert_end(6);

    // Display original list
    cout << "Original: ";
    display();

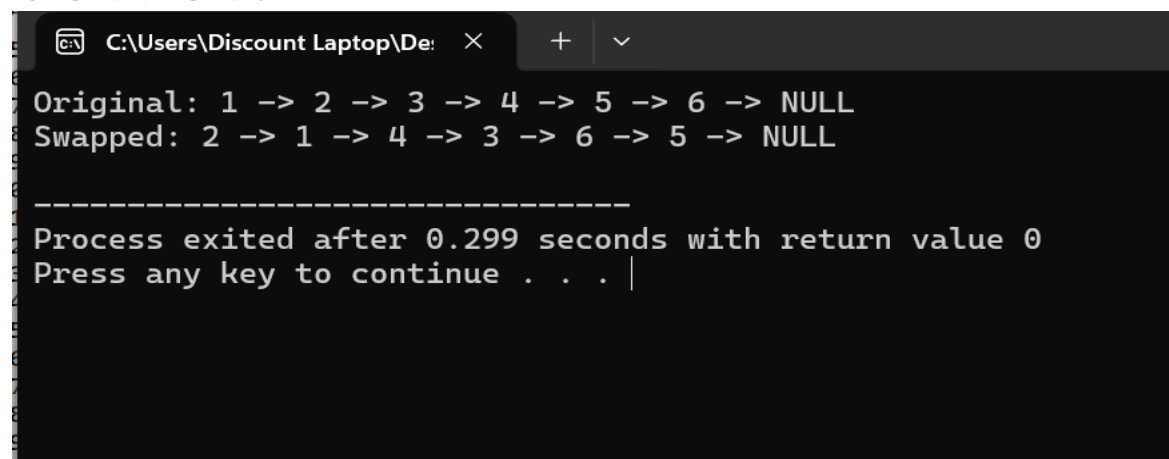
    // Swap every two consecutive nodes
    swapPairs();

    // Display swapped list
    cout << "Swapped: ";
    display();

    return 0;    // End the program
}

```

OUTPUT:



```

C:\Users\Discount Laptop\De:  X  +  v
Original: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> NULL
Swapped: 2 -> 1 -> 4 -> 3 -> 6 -> 5 -> NULL

-----
Process exited after 0.299 seconds with return value 0
Press any key to continue . . . |

```

DOUBLY LINK LIST

TASK 01

CODE

```
#include <iostream>
#include <string>
using namespace std;

struct NodeType {
    string name;
    NodeType *next;
    NodeType *prev;
};

NodeType *first = NULL, *last = NULL;

void insert_end(string val) {
    NodeType *p = new NodeType;
    p->name = val;
    p->next = NULL;
    p->prev = NULL;

    if (first == NULL) {
        first = last = p;
    } else {
        last->next = p;
        p->prev = last;
        last = p;
    }
}

// Swap values from both ends toward center
void swapEndsTowardsCenter() {
    if (first == NULL || first == last) return;

    NodeType *left = first;
    NodeType *right = last;

    while (left != right && left->prev != right) {
        swap(left->name, right->name);
        left = left->next;
        right = right->prev;
    }
}

void display() {
    NodeType *p = first;
    while (p != NULL) {
        cout << p->name;
        if (p->next != NULL) cout << " <-> ";
        p = p->next;
    }
}
```

```

    }
    cout << endl;
}

int main() {
    insert_end("Alice"); insert_end("Bob"); insert_end("Charlie");
    insert_end("Dana"); insert_end("Eva"); insert_end("Frank");

    cout << "Original: "; display();
    swapEndsTowardsCenter();
    cout << "After: "; display();

    return 0;
}

```

OUTPUT:

```

C:\Users\Discount Laptop\De...
Original: Alice <-> Bob <-> Charlie <-> Dana <-> Eva <-> Frank
After: Frank <-> Eva <-> Dana <-> Charlie <-> Bob <-> Alice

-----
Process exited after 0.2943 seconds with return value 0
Press any key to continue . . . |

```

TASK 02

CODE

```

#include <iostream>
#include <string>
using namespace std;
// structure for song node
struct Song {
    string title;
    string artist;
    Song *next;
    Song *prev;
};
// pointers for 1 last and current song
Song *head = NULL, *tail = NULL, *current = NULL;

void addSong(string t, string a) {
    //song node created
    Song *p = new Song;
    p->title = t;
    p->artist = a;
    p->next = NULL;
}

```

```

    p->prev = NULL;
    // if playlist is empty
    if (head == NULL) {
        head = tail = current = p;
    } else {
// connect old song node to the new song
        tail->next = p;
// connect new song back to old
        p->prev = tail;
// update tail
        tail = p;
    }
}
// function to play next song
void nextSong() {
    if (current == NULL) { cout << "Playlist empty\n"; return; }
    if (current->next == NULL) current = head;
// back to first song
    else current = current->next;
// display current song
    cout << "Now playing: " << current->title << " by " << current->artist << endl;
}
// func for previous song
void prevSong() {
// check if list is empty
    if (current == NULL) { cout << "Playlist empty\n"; return; }
//if current song is first song
    if (current->prev == NULL) current = tail;
// go to last song
    else current = current->prev;
    cout << "Now playing: " << current->title << " by " << current->artist << endl;
}
//func to display first to last
void displayForward() {
    cout << "Playlist (forward): ";
    Song *p = head;
    while (p != NULL) {
        cout << p->title;
        if (p->next != NULL) cout << " -> ";
        p = p->next;
    }
    cout << endl;
}
// func to display last to first
void displayBackward() {
    cout << "Playlist (backward): ";
    Song *p = tail;
    while (p != NULL) {
        cout << p->title;
        if (p->prev != NULL) cout << " <- ";
    }
}

```

```

        p = p->prev;
    }
    cout << endl;
}
// func to remove song
void removeSong(string title) {
    Song *p = head;
    while (p != NULL) {
        if (p->title == title) {
            if (current == p)
                current = (p->next != NULL) ? p->next : p->prev;

            if (p->prev != NULL) p->prev->next = p->next;
            else head = p->next;

            if (p->next != NULL) p->next->prev = p->prev;
            else tail = p->prev;

            delete p;
            cout << "Removed: " << title << endl;
            return;
        }
        p = p->next;
    }
    cout << "Song not found: " << title << endl;
}

int main() {
    addSong("Song A", "Artist 1");
    addSong("Song B", "Artist 2");
    addSong("Song C", "Artist 3");
    addSong("Song D", "Artist 4");

    displayForward();
    displayBackward();

    cout << "\n--- Playback ---" << endl;
    nextSong();
    nextSong();
    prevSong();

    cout << "\n--- Remove ---" << endl;
    removeSong("Song B");
    displayForward();

    return 0;
}

```

OUTPUT:

```
C:\Users\Discount Laptop\De:  X  +  v
Playlist (forward): Song A -> Song B -> Song C -> Song D
Playlist (backward): Song D <- Song C <- Song B <- Song A

--- Playback ---
Now playing: Song B by Artist 2
Now playing: Song C by Artist 3
Now playing: Song B by Artist 2

--- Remove ---
Removed: Song B
Playlist (forward): Song A -> Song C -> Song D

-----
Process exited after 0.6399 seconds with return value 0
Press any key to continue . . . |
```

TASK 03

CODE

```
#include <iostream>
#include <string>
using namespace std;

// Item node (bottom level)
struct Item {
    string name;
    int quantity;
    Item *next;
};

// Section node (middle level)
struct Section {
    string sectionName;
    Section *next;
    Item *itemsHead;
};

// Store node (top level)
struct Store {
    string storeName;
    Store *next;
    Section *sectionsHead;
};

Store *storesHead = NULL;

Store* findStore(string name) {
    Store *p = storesHead;
    while (p != NULL) {
        if (p->storeName == name) return p;
        p = p->next;
    }
}
```

```

    return NULL;
}

Section* findSection(Store *store, string name) {
    if (store == NULL) return NULL;
    Section *p = store->sectionsHead;
    while (p != NULL) {
        if (p->sectionName == name) return p;
        p = p->next;
    }
    return NULL;
}

// Add a new store
void addStore(string name) {
    Store *p = new Store;
    p->storeName = name;
    p->next = NULL;
    p->sectionsHead = NULL;

    if (storesHead == NULL) storesHead = p;
    else {
        Store *temp = storesHead;
        while (temp->next != NULL) temp = temp->next;
        temp->next = p;
    }
    cout << "Store added: " << name << endl;
}

// Add a new section in a store
void addSection(string storeName, string sectionName) {
    Store *store = findStore(storeName);
    if (store == NULL) { cout << "Store not found\n"; return; }

    if (findSection(store, sectionName) != NULL) {
        cout << "Section already exists\n";
        return;
    }

    Section *p = new Section;
    p->sectionName = sectionName;
    p->next = NULL;
    p->itemsHead = NULL;

    if (store->sectionsHead == NULL) store->sectionsHead = p;
    else {
        Section *temp = store->sectionsHead;
        while (temp->next != NULL) temp = temp->next;
        temp->next = p;
    }
}

```

```

    cout << "Section added: " << sectionName << " in " << storeName << endl;
}

// Store an item in a particular section of a particular store
void addItem(string storeName, string sectionName, string itemName, int qty) {
    Store *store = findStore(storeName);
    if (store == NULL) { cout << "Store not found\n"; return; }

    Section *section = findSection(store, sectionName);
    if (section == NULL) { cout << "Section not found\n"; return; }

    Item *p = new Item;
    p->name = itemName;
    p->quantity = qty;
    p->next = NULL;

    if (section->itemsHead == NULL) section->itemsHead = p;
    else {
        Item *temp = section->itemsHead;
        while (temp->next != NULL) temp = temp->next;
        temp->next = p;
    }
    cout << "Item added: " << itemName << " (" << qty << ") in "
        << storeName << " -> " << sectionName << endl;
}

// Remove an item from a particular section of a particular store
void removeItem(string storeName, string sectionName, string itemName) {
    Store *store = findStore(storeName);
    if (store == NULL) { cout << "Store not found\n"; return; }

    Section *section = findSection(store, sectionName);
    if (section == NULL) { cout << "Section not found\n"; return; }

    Item *current = section->itemsHead;
    Item *prev = NULL;

    while (current != NULL) {
        if (current->name == itemName) {
            if (prev == NULL) section->itemsHead = current->next;
            else prev->next = current->next;
            delete current;
            cout << "Item removed: " << itemName << endl;
            return;
        }
        prev = current;
        current = current->next;
    }
    cout << "Item not found: " << itemName << endl;
}

```

```

// Display list of all items of a particular section of a store
void displaySectionItems(string storeName, string sectionName) {
    Store *store = findStore(storeName);
    if (store == NULL) { cout << "Store not found\n"; return; }

    Section *section = findSection(store, sectionName);
    if (section == NULL) { cout << "Section not found\n"; return; }

    cout << "Items in " << storeName << " -> " << sectionName << ": ";
    Item *p = section->itemsHead;
    while (p != NULL) {
        cout << p->name << "(" << p->quantity << ") ";
        p = p->next;
    }
    cout << endl;
}

```

```

// Display list of items for a given store
void displayStoreItems(string storeName) {
    Store *store = findStore(storeName);
    if (store == NULL) { cout << "Store not found\n"; return; }

    cout << "Store: " << storeName << endl;
    Section *s = store->sectionsHead;
    while (s != NULL) {
        cout << " Section " << s->sectionName << ": ";
        Item *p = s->itemsHead;
        while (p != NULL) {
            cout << p->name << "(" << p->quantity << ") ";
            p = p->next;
        }
        cout << endl;
        s = s->next;
    }
}

```

```

int main() {
    addStore("Walmart");
    addStore("Target");

    addSection("Walmart", "Toys");
    addSection("Walmart", "Grocery");
    addSection("Target", "Fruits");

    addItem("Walmart", "Toys", "Lego", 50);
    addItem("Walmart", "Toys", "Doll", 30);
    addItem("Walmart", "Grocery", "Milk", 100);
    addItem("Target", "Fruits", "Apple", 200);
}

```

```

displaySectionItems("Walmart", "Toys");
displayStoreItems("Walmart");

removeItem("Walmart", "Toys", "Doll");
displaySectionItems("Walmart", "Toys");

return 0;
}

```

OUTPUT:

```

Store added: Walmart
Store added: Target
Section added: Toys in Walmart
Section added: Grocery in Walmart
Section added: Fruits in Target
Item added: Lego (50) in Walmart -> Toys
Item added: Doll (30) in Walmart -> Toys
Item added: Milk (100) in Walmart -> Grocery
Item added: Apple (200) in Target -> Fruits
Items in Walmart -> Toys: Lego(50) Doll(30)
Store: Walmart
    Section Toys: Lego(50) Doll(30)
    Section Grocery: Milk(100)
Item removed: Doll
Items in Walmart -> Toys: Lego(50)

-----
Process exited after 0.4114 seconds with return value 0
Press any key to continue . . . |

```

TASK 04

CODE:

```

#include <iostream>
using namespace std;

struct NodeType {
    int data;
    NodeType *next;
    NodeType *prev;
};

NodeType *first = NULL, *last = NULL;

```

```

void insert_end(int val) {
    NodeType *p = new NodeType;
    p->data = val;
    p->next = NULL;
    p->prev = NULL;
    if (first == NULL) first = last = p;
    else { last->next = p; p->prev = last; last = p; }
}

// Pattern: 1 -> 8 -> 3 -> 6 -> 5 -> 4 -> 7 -> 2 -> 9
// First and last remain fixed; alternate swaps toward center
void specialPattern() {
    if (first == NULL || first == last) return;

    NodeType *left = first->next;    // start after first
    NodeType *right = last->prev;    // start before last

    bool swapFlag = true;

    while (left != right && left->prev != right) {
        if (swapFlag) {
            swap(left->data, right->data);
            left = left->next;
            right = right->prev;
        } else {
            left = left->next;
            right = right->prev;
        }
        swapFlag = !swapFlag;
    }
}

void display() {
    NodeType *p = first;
    while (p != NULL) {
        cout << p->data;
        if (p->next != NULL) cout << " -> ";
        p = p->next;
    }
    cout << endl;
}

int main() {
    for (int i = 1; i <= 9; i++) insert_end(i);

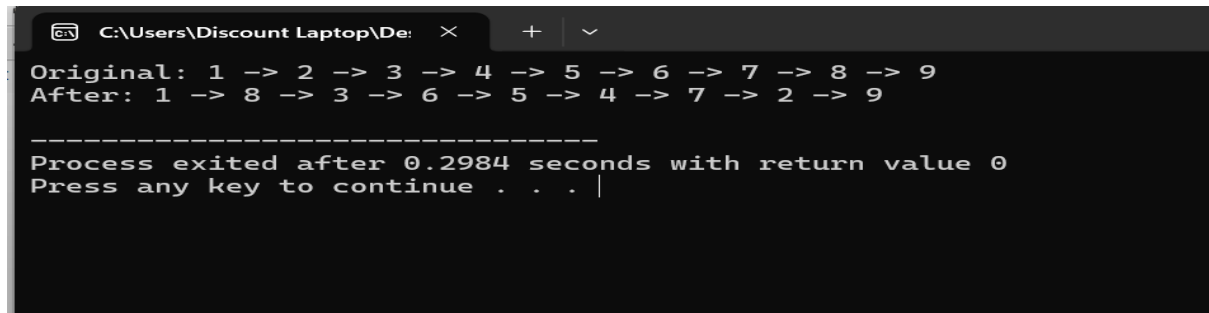
    cout << "Original: "; display();
    specialPattern();
    cout << "After: "; display();

    return 0;
}

```

```
}
```

OUTPUT:



```
C:\Users\Discount Laptop\De:  X  +  v

Original: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9
After: 1 -> 8 -> 3 -> 6 -> 5 -> 4 -> 7 -> 2 -> 9

-----
Process exited after 0.2984 seconds with return value 0
Press any key to continue . . . |
```

CIRCULAR LINK LIST

TASK 01

CODE

```
#include <iostream>
using namespace std;

struct NodeType {
    int data;
    NodeType *next;
};

NodeType *pNode = NULL;

// Insert into circular linked list (as per Lab 04 Activity 1)
void insert(int value) {
    NodeType *p = new NodeType;
    p->data = value;

    if (pNode == NULL) {
        p->next = p;
        pNode = p;
    } else {
        p->next = pNode->next;
        pNode->next = p;
        pNode = p;
    }
}

// Josephus Problem Solution
int josephus(int N, int M) {
    pNode = NULL;

    // Create circle with N persons numbered 1 to N
    for (int i = 1; i <= N; i++) {
        insert(i);
    }
}
```

```

    }

    // Start from first person
    NodeType *current = pNode->next; // points to person 1
    NodeType *prev = pNode;         // points to person N

    cout << "Elimination order: ";

    // Eliminate until only one remains
    while (current->next != current) {
        // Skip M-1 persons
        for (int count = 1; count < M; count++) {
            prev = current;
            current = current->next;
        }

        // Kill Mth person
        cout << current->data << " ";
        prev->next = current->next;

        // If current was pNode, update pNode
        if (current == pNode) {
            pNode = prev;
        }

        delete current;
        current = prev->next;
    }

    cout << endl;
    int survivor = current->data;
    delete current;
    pNode = NULL;
    return survivor;
}

int main() {
    int N, M;
    cout << "Enter total number of persons (N): ";
    cin >> N;
    cout << "Enter skip count (M): ";
    cin >> M;

    int safePosition = josephus(N, M);
    cout << "Safe position (survivor): " << safePosition << endl;

    return 0;
}

```

OUTPUT:

```
C:\Users\Discount Laptop\De: × + ∨
Enter total number of persons (N): 7
Enter skip count (M): 3
Elimination order: 3 6 2 7 5 1
Safe position (survivor): 4

-----
Process exited after 3.043 seconds with return value 0
Press any key to continue . . . |
```

TASK 02

TASK SCHEDULING SYSTEM

CODE

```
#include <iostream>
#include <string>
using namespace std;

// TASK SCHEDULING SYSTEM USING CIRCULAR LINKED LIST
// Each node represents a task with name, priority, and status.
// The list is circular (last node points back to first node).

// Structure to represent a Task (node of circular linked list)
struct Task {
    string name;      // Name of the task (e.g., "Design")
    int priority;     // Priority of the task (lower number = higher priority)
    string status;    // "pending", "in-progress", or "completed"
    Task *next;       // Pointer to the next task in the circular list
};

// Global pointers
Task *head = NULL;   // Points to the first task in the circular list
Task *currentTask = NULL; // Tracks the current task for round-robin execution

// FUNCTION: addTask
// Adds a new task at the END of the circular linked list.
// Ensures the list remains circular (last node -> head).
void addTask(string name, int priority) {
    // Step 1: Allocate memory for the new task node
    Task *p = new Task;
    p->name = name;
    p->priority = priority;
    p->status = "pending"; // Every new task starts as "pending"

    // Step 2: Check if list is empty
```

```

if (head == NULL) {
    // First node: points to itself (single-node circular list)
    p->next = p;
    head = p;
} else {
    // Step 3: Traverse to the last node (whose next is head)
    Task *temp = head;
    while (temp->next != head) {
        temp = temp->next;
    }
    // Step 4: Link new node at the end and reconnect to head
    temp->next = p;    // Old last node points to new node
    p->next = head;    // New node points back to head (keeps circular)
}
cout << "Task added: " << name << endl;
}

```

```

// FUNCTION: removeTask
// Removes a task by its name from the circular linked list.
// Handles 3 cases:
// (1) Only one node in the list
// (2) Deleting the head node
// (3) Deleting a middle or last node
void removeTask(string name) {
    // Check if list is empty
    if (head == NULL) {
        cout << "List is empty." << endl;
        return;
    }
}

```

```

Task *current = head; // Pointer to traverse the list
Task *prev = NULL;    // Pointer to keep track of previous node

```

```

// Find the last node (needed when deleting head)
Task *last = head;
while (last->next != head) {
    last = last->next;
}

```

```

// Traverse the circular list using do-while
do {
    if (current->name == name) {
        // Case 1: Only one node
        if (current == head && head->next == head) {
            delete head;
            head = NULL;
            currentTask = NULL;
            cout << "Task removed: " << name << endl;

```

```

        return;
    }
    // Case 2: Deleting head node
    head = head->next;    // Move head to next node
    last->next = head;    // Last node points to new head
    if (currentTask == current) {
        currentTask = head; // Update round-robin pointer if needed
    }
    delete current;
    cout << "Task removed: " << name << endl;
    return;
}
// -- Case 3: Middle or last node
else {
    prev->next = current->next; // Bypass the deleted node
    if (currentTask == current) {
        currentTask = current->next; // Move round-robin pointer
    }
    delete current;
    cout << "Task removed: " << name << endl;
    return;
}
}
// Move to next node
prev = current;
current = current->next;
} while (current != head);

// Task not found
cout << "Task not found: " << name << endl;
}

// FUNCTION: getNextTask
// Returns the next pending task in round-robin order.
// Skips tasks whose status is "completed".
void getNextTask() {
    // Check if list is empty
    if (head == NULL) {
        cout << "No tasks available." << endl;
        return;
    }

    // Move currentTask to the next task (round-robin)
    if (currentTask == NULL) {
        currentTask = head;    // First call: start from head
    } else {
        currentTask = currentTask->next; // Subsequent calls: next task
    }

    // Skip completed tasks

```

```

Task *start = currentTask; // Remember where we started
while (currentTask->status == "completed") {
    currentTask = currentTask->next;
    // If we came back to start, everyone is completed
    if (currentTask == start) {
        cout << "All tasks are completed!" << endl;
        return;
    }
}

// Display the found pending task
cout << "Next task: " << currentTask->name
    << " (Priority: " << currentTask->priority
    << ", Status: " << currentTask->status << ")" << endl;
}

// FUNCTION: displayAllTasks
// Displays all tasks in the circular list.
// Uses do-while so we traverse until we reach head again.
void displayAllTasks() {
    if (head == NULL) {
        cout << "No tasks in the list." << endl;
        return;
    }

    cout << "\n--- All Tasks ---" << endl;
    Task *temp = head;
    do {
        cout << "Name: " << temp->name
            << " | Priority: " << temp->priority
            << " | Status: " << temp->status << endl;
        temp = temp->next;
    } while (temp != head); // Stop when we return to head
    cout << "-----" << endl;
}

// FUNCTION: updateTaskStatus
// Updates the status of a task found by its name.
void updateTaskStatus(string name, string newStatus) {
    if (head == NULL) {
        cout << "List is empty." << endl;
        return;
    }

    Task *temp = head;
    do {
        if (temp->name == name) {
            temp->status = newStatus; // Update the status
            cout << "Task '" << name << "' status updated to: " << newStatus << endl;
        }
    } while (temp != head);
}

```

```

        return;
    }
    temp = temp->next;
} while (temp != head);

cout << "Task not found: " << name << endl;
}

```

```

// MAIN FUNCTION — Demonstrates all operations
int main() {
    // Add 4 tasks
    addTask("Design", 1);
    addTask("Coding", 2);
    addTask("Testing", 3);
    addTask("Deployment", 4);

    // Show all tasks
    displayAllTasks();

    // Round-robin demonstration
    cout << "\n--- Round Robin ---" << endl;
    getNextTask(); // Design
    getNextTask(); // Coding

    // Mark Coding as completed
    updateTaskStatus("Coding", "completed");

    getNextTask(); // Testing
    getNextTask(); // Deployment
    getNextTask(); // Design (wraps around, Coding is skipped)

    // Remove Testing
    cout << "\n--- Remove Testing ---" << endl;
    removeTask("Testing");
    displayAllTasks();

    return 0;
}

```

OUTPUT:

```
Task added: Design
Task added: Coding
Task added: Testing
Task added: Deployment

--- All Tasks ---
Name: Design | Priority: 1 | Status: pending
Name: Coding | Priority: 2 | Status: pending
Name: Testing | Priority: 3 | Status: pending
Name: Deployment | Priority: 4 | Status: pending
-----

--- Round Robin ---
Next task: Design (Priority: 1, Status: pending)
Next task: Coding (Priority: 2, Status: pending)
Task 'Coding' status updated to: completed
Next task: Testing (Priority: 3, Status: pending)
Next task: Deployment (Priority: 4, Status: pending)
Next task: Design (Priority: 1, Status: pending)

--- Remove Testing ---
Task removed: Testing

--- All Tasks ---
Name: Design | Priority: 1 | Status: pending
Name: Coding | Priority: 2 | Status: completed
Name: Deployment | Priority: 4 | Status: pending
-----
```